Models of Computation (TC-MC)

Finite-state Automata
and
Regular Languages

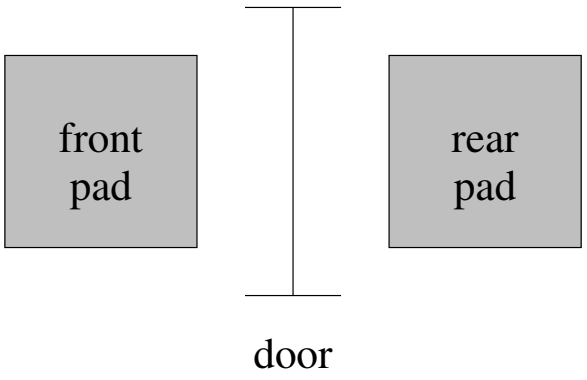
Stefan Dantchev

Durham University Department of Computer Science

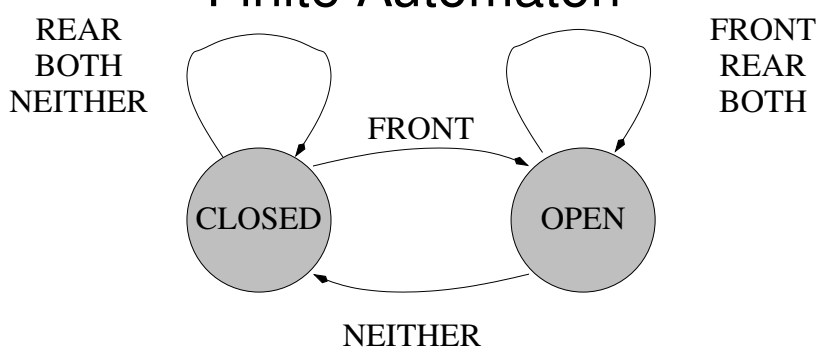
Deterministic Finite-state Automaton (DFA)

Motivating Example

Simple Automatic Door



Finite Automaton



	NEITHER	FRONT	REAR	BOTH
CLOSED	CLOSED	OPEN	CLOSED	CLOSED
OPEN	CLOSED	OPEN	OPEN	OPEN

Abstract Example

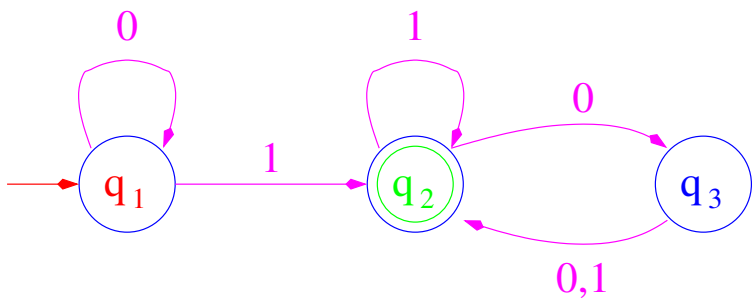


Figure 1: Accepts the strings that ends in an even number of 0's following the last 1

Concepts we could identify on the **state diagram** above: states, start state, accept state(s), transitions.

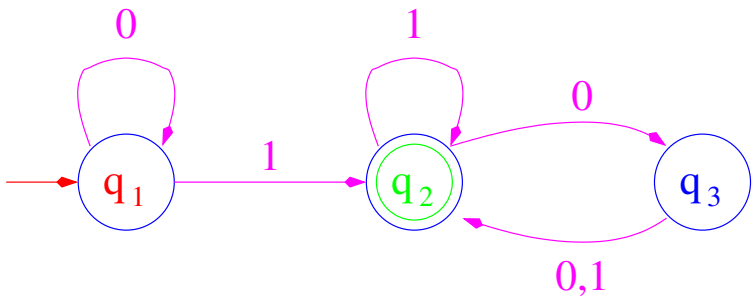
Formal Definition

Definition. (Deterministic Finite-state Automaton) *A DFA is a* 5*-tuple* $(Q, \Sigma, \delta, q_0, F)$*, where*

1. Q *is a finite set of* ***states****,*
2. Σ *is a finite* ***alphabet****,*
3. $\delta : Q \times \Sigma \to Q$ *is the* ***transition function****,*
4. $q_0 \in Q$ *is* ***the start state****, and*
5. $F \subseteq Q$ *is the set of* ***accept states****.*

Definition: example

State diagram



$M = \{Q, \Sigma, \delta, q_1, F\}$

Formal description

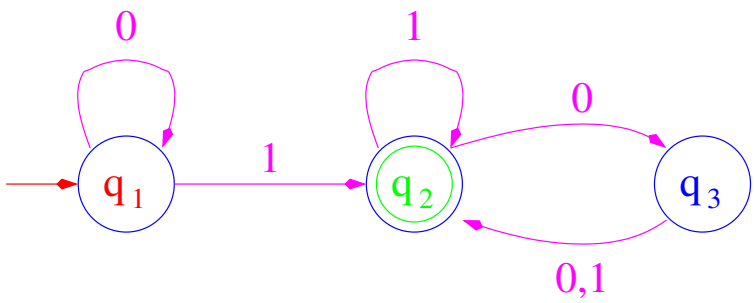
1. $Q = \{q_1, q_2, q_3\}$,
2. $\Sigma = \{0, 1\}$,
3. δ is described as

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

4. q_1 **is the start state**, and
5. $F = \{q_2\}$

Computation: example and formal definition

State diagram



How does it work on different inputs?

1. 0100: $q_1 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \xrightarrow{0} q_3 \xrightarrow{0} q_2$ accept

2. 1010: $q_1 \xrightarrow{1} q_2 \xrightarrow{0} q_3 \xrightarrow{1} q_2 \xrightarrow{0} q_3$ reject

Formal definition

Definition. *Let* $M = (Q, \Sigma, \delta, q_0, F)$ *be a DFA and* $w = w_1 w_2 \ldots w_n$ *be a word over* Σ. M ***accepts*** w *if there is a sequence of states* $r_0, r_1, r_2, \ldots r_n$ *satisfying the following conditions:*

1. $r_0 = q_0$,

2. $\delta(r_i, w_{i+1}) = r_{i+1}$ *for every* i, $0 \leq i \leq n-1$, *and*

3. $r_n \in F$.

Regular Languages

Definition. *The DFA M* ***recognises*** *the language L if $L = \{w \,|\, M \text{ accepts } w\}$.*

Definition. (Regular Language) *A language is called a* ***regular language*** *if some DFA recognises it.*

Designing DFA

1. It is a creative process, and is very similar to writing a computer program.

2. There are no recipes. Looking at many examples helps.

Example 2

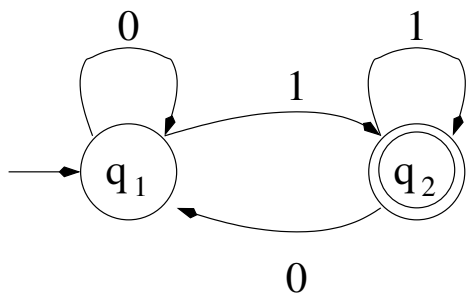


Figure 2: Accepts the strings ending in a 1

Example 3

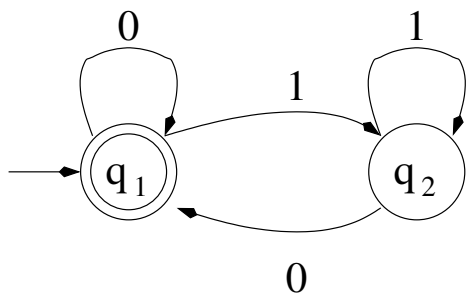


Figure 3: Accepts the strings ending in a 0 as well as the empty string ε

Example 4

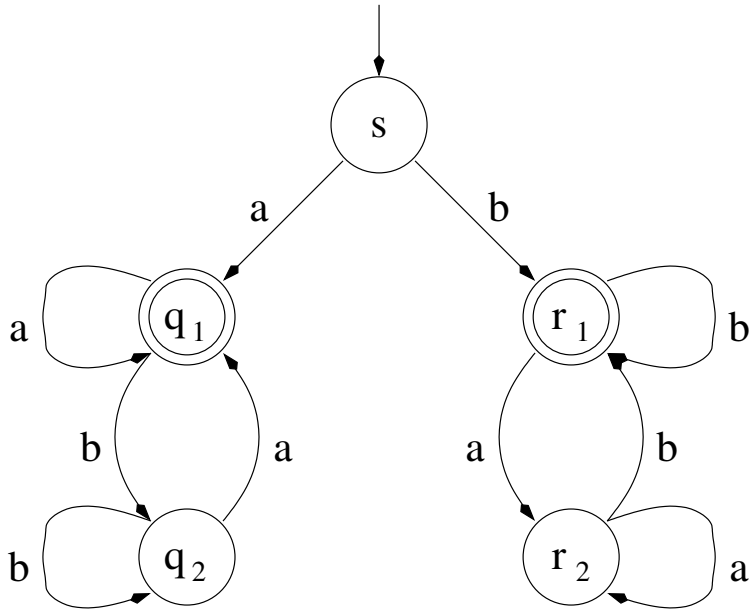


Figure 4: Accepts the strings starting and ending with the same symbol

Example 5

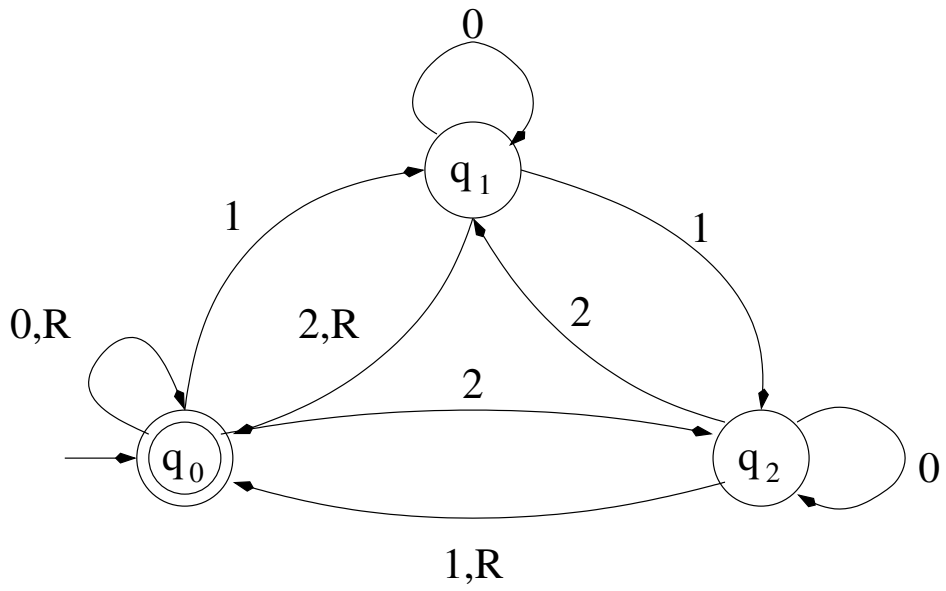


Figure 5: Counts modulo 3 (R resets the counter)

Regular Expression

Regular Operations

- Boolean (set-theoretic):

union	$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$
intersection	$A \cap B = \{x \mid x \in A \text{ and } x \in B\}$
difference	$A \setminus B = \{x \mid x \in A \text{ and } x \notin B\}$
complement	$\overline{A} = \Sigma^* \setminus A$

- Language theory specific:

concatenation	$A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$
star	$A^* = \{x_1 x_2 \ldots x_k \mid k \geq 0 \text{ and } x_i \in A \text{ for every } i, 1 \leq i \leq k\}$
alternatively	$A^* = \bigcup_{i=0}^{\infty} A^i$

Regular Expression (RE)

A **Regular Expression (RE)** R **defines** a (*regular?*) **language** $L(R)$. We shall eventually prove that RE$\equiv$DFA (i.e. REs define exactly class of the regular languages).

The definition is inductive (recursive), i.e. there are **initial RE**, and a **new REs can be obtained** from old ones **by means of Regular Operations**.

Definition. *(RE) R is a Regular Expression over the alphabet Σ if R is*

1. *a for some $a \in \Sigma$,*
2. *ε,*
3. *$\emptyset$,*
4. *$(R_1 \cup R_2)$, where R_1 and R_2 are REs,*
5. *$(R_1 \circ R_2)$, where R_1 and R_2 are REs, or*
6. *(R_1^*), where R_1 is a RE.*

Conventions: Concatenation symbol may be omitted, i.e. R_1R_2 means $R_1 \circ R_2$. Parentheses may be omitted, bearing in mind that the **precedence order** (highest to lowest) is *, $\circ$, $\cup$, i.e. $01 \cup 1^*$ means $((01) \cup (1^*))$.

Combining automata

Want to implement an operation on languages by combining the corresponding automata.

Given L_1 recognised by $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and L_2 recognised by $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$; want to combine M_1 and M_2 into a *new automaton* M that would recognise $L_1 \cup L_2$.

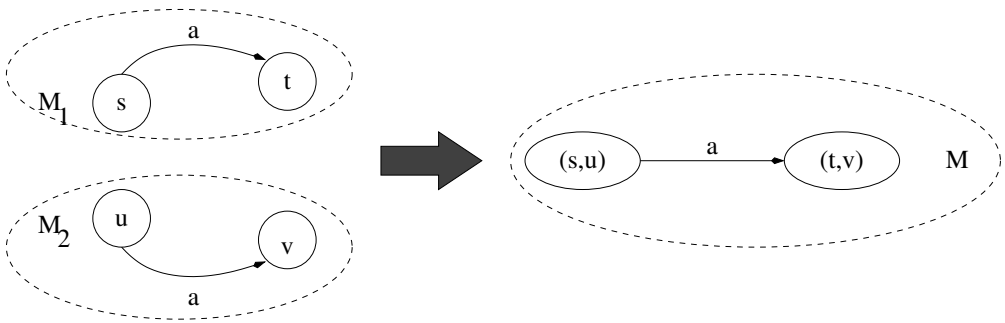
Naive idea: Simulate first M_1 on the input and then simulate M_2 on the same input; accept if either M_1 or M_2 or both accept. Does not work! After M_1 has run on the input, the input is exhausted; there is no way to "rewind" it in order to run M_2.

Can we make this idea work at all!?

Combining automata

Run M_1 and M_2 on the input in parallel!
How do we do it?

Use pairs of states (a state for each DFA).
Define the transition function as shown below.



Formally, the components of the new automaton $M = (Q, \Sigma, \delta, q_0, F)$ are

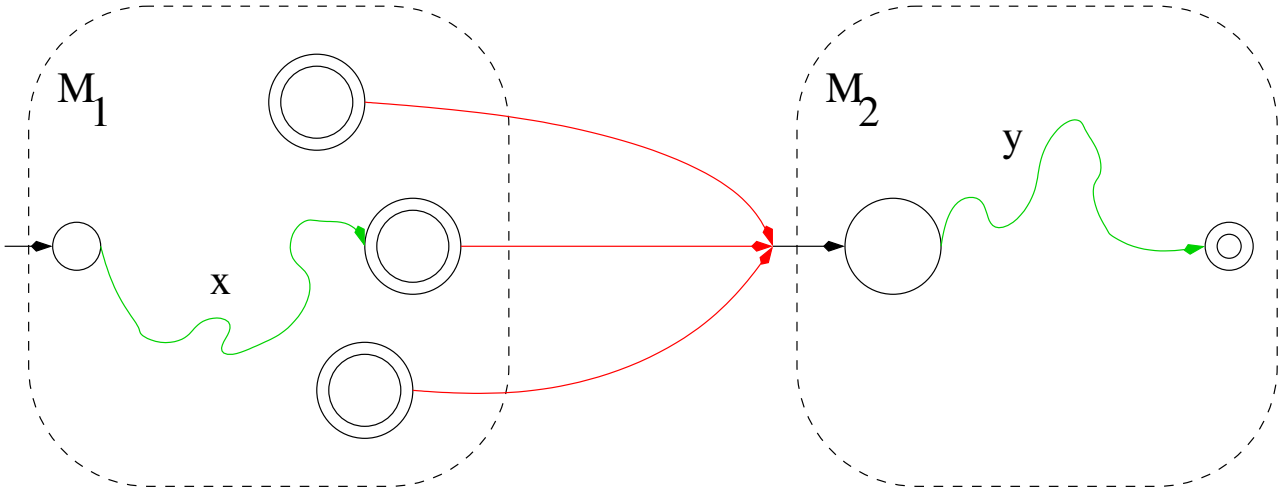
1. $Q = Q_1 \times Q_2$,
2. $\delta((s,u),a) = (\delta_1(s,a), \delta_2(u,a))$ for every $s \in Q_1$, $u \in Q_2$, $a \in \Sigma$,
3. $q_0 = (q_1, q_2)$, and
4. $F = \{(s,u) \in Q \,|\, s \in F_1 \text{ or } u \in F_2\}$

Concatenation

$w \in L_1 \circ L_2$ only if there are words x and y such that $w = xy$, $x \in L_1$ and $y \in L_2$.

We need to run M_1on a prefix of w and then to run M_2 on the rest. How do we find the break-point?

Guess it (***non-deterministically***)!



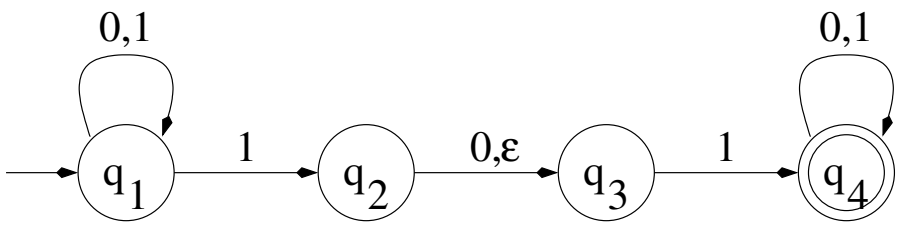
The result is not a DFA because of the red transitions!

Non-deterministic Finite Automaton (NFA)

Definition. (Non-deterministic Finite-state Automaton) *A NFA is a* 5*-tuple* $(Q, \Sigma, \delta, q_0, F)$*, where*

1. Q *is a finite set of* ***states****,*
2. Σ *is a finite* ***alphabet****,*
3. $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathscr{P}(Q)$ *is the* ***transition function****,*
4. $q_0 \in Q$ *is* ***the start state****, and*
5. $F \subseteq Q$ *is the set of* ***accept states****.*

Example



NFA that accepts the words containing 11 or 101 as a sub-word

Three potential problems:

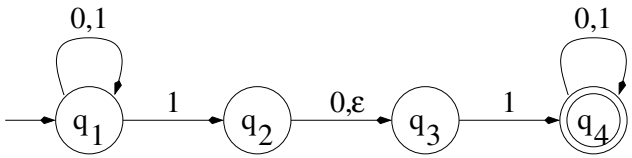
1. More than one next possible state (in q_1 read 1).

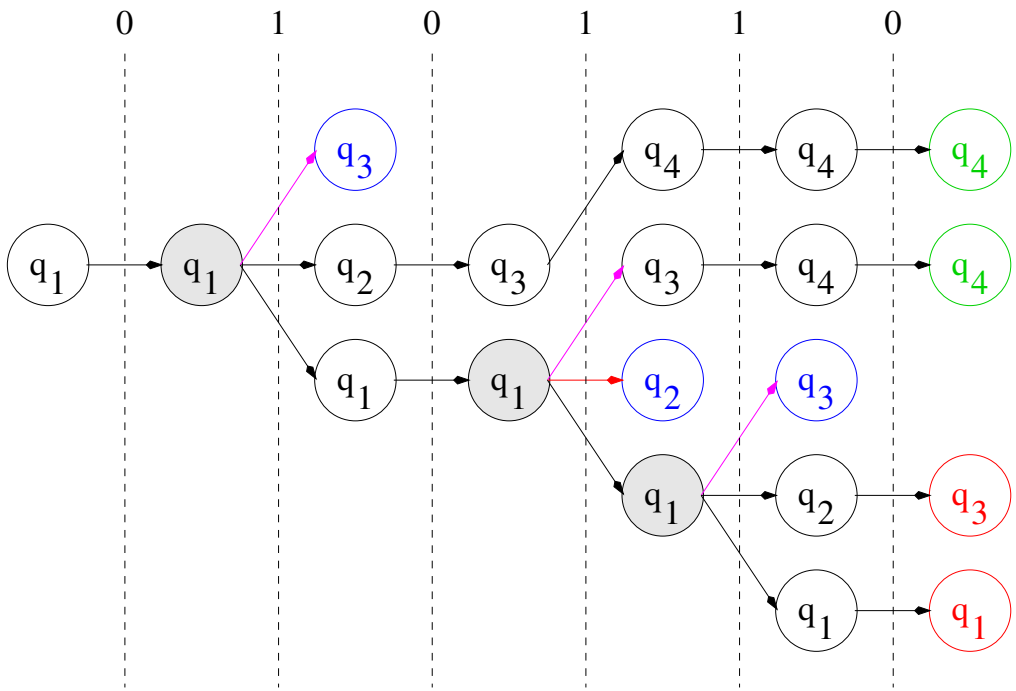
2. ε-transition (in q_2, and the next symbol is 0).

3. No "next" state (in q_3 read 0): *the computation hangs, i.e. ends before the input has been exhausted*.

Warning: ε is not a letter from the alphabet Σ; it means "go straight away to that state and postpone reading the next input symbol"!

Example





All the possible computations on the input 010110

Important convention: a *single step* in a computation consists of *first reading the next input symbol* followed by *taking any number of ε-transition*.

Computation of a NFA

Definition. *Let* $M = (Q, \Sigma, \delta, q_0, F)$ *be a NFA and* $w = w_1 w_2 \ldots w_n$ *be a word over* $\Sigma \cup \{\varepsilon\}$.

Important remark: ε's can be *freely added* inside the actual word $\in \Sigma^*$, e.g. 010110 can be viewed as, say, $\varepsilon 01 \varepsilon \varepsilon 01 \varepsilon 10$.

Definition. M ***accepts*** w *if there exists a sequence of states* $r_0, r_1, r_2, \ldots r_n$ *satisfying the following conditions:*

1. $r_0 = q_0$,
2. $r_{i+1} \in \delta(r_i, w_{i+1})$ *for every* i, $0 \leq i \leq n-1$, *and*
3. $r_n \in F$.

Computation of a NFA

DFA: There is a unique computation (path). It either accepts or rejects.

NFA: There are, in general, many computational paths. The NFA accepts if at least one computation accepts. The NFA rejects only if all the computations reject.

Two ways to understand ***non-deterministic computation***:

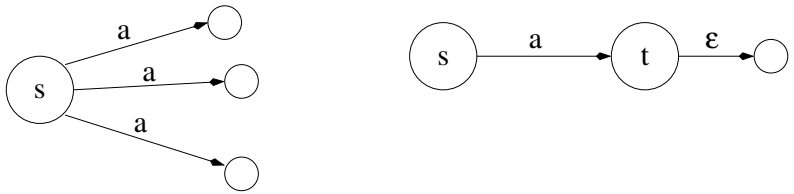
try all possible paths and accept iff at least one accepts

or

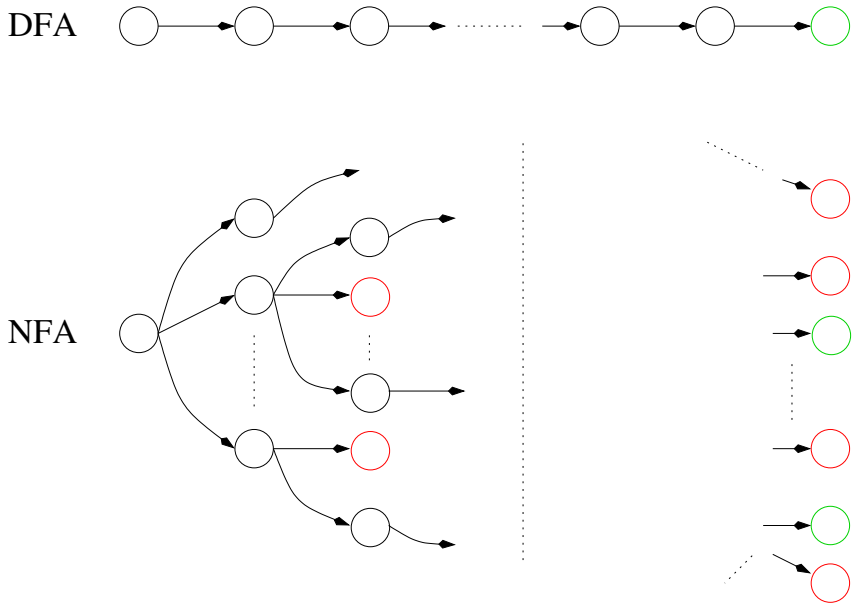
always make the "right" choice if there was one.

Equivalence of NFAs and DFAs

NFA vs DFA



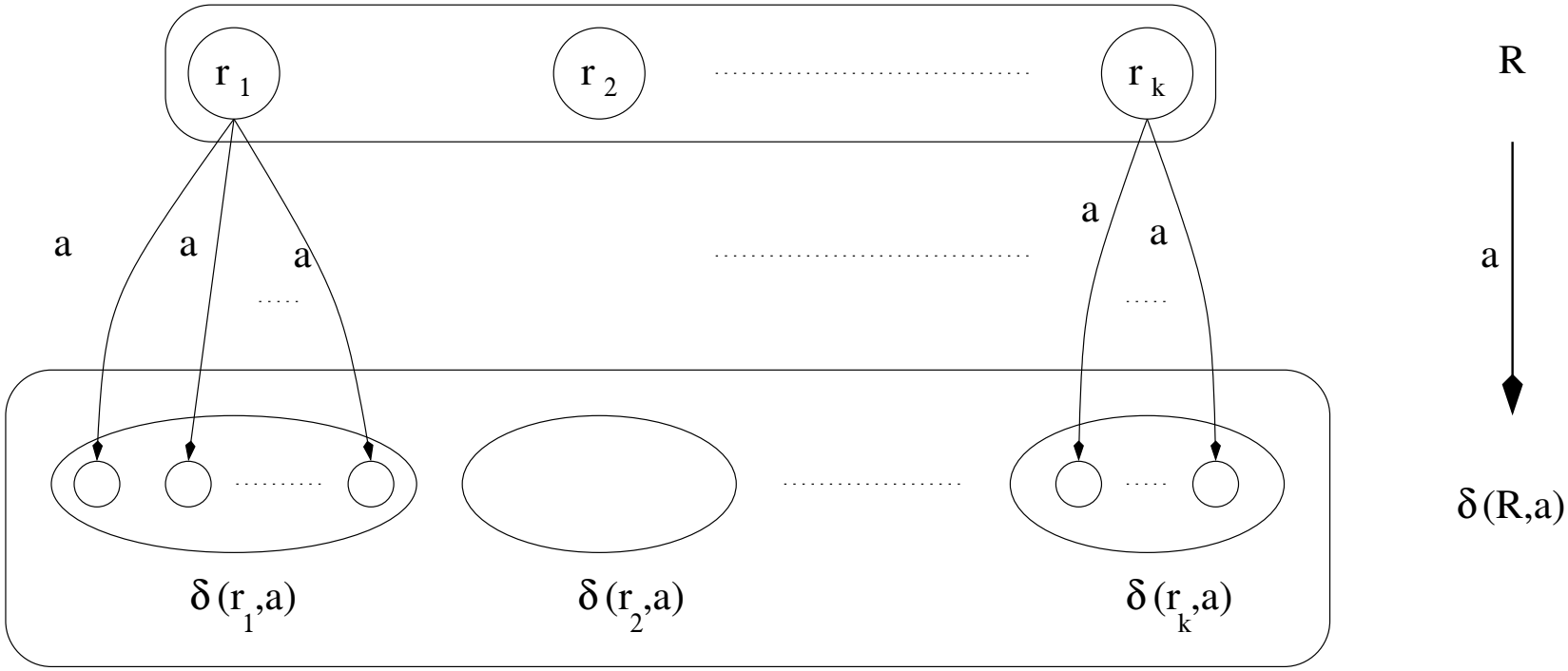
Two sources of non-determinism in a NFA



Accepting computation(s)

Converting a NFA into a DFA

The idea behind the construction: At any time there are *a number of possible states* the NFA computation can be into. As the DFA needs to keep track of all these, the DFA's states will correspond to subsets of the NFA's-state set. The transition function of the DFA can then be defined as shown on the figure:



Question: Do we need the empty set as a state of the DFA and if so, why? What are the outgoing transitions for the empty-set state in the DFA?

Converting a NFA into a DFA

Formally: Let $N = (Q, \Sigma, \delta, q_0, F)$ be a NFA recognising some language L. (Assume than N has no ε-transitions - we will fix that later.). We shall construct a DFA $M = (Q', \Sigma, \delta', q_0', F')$ that recognises the same language L as follows.

1. $Q' = \mathscr{P}(Q)$.

2. For $R \in Q'$ and $a \in \Sigma$ let

$$\delta'(R, a) = \{q \in Q \,|\, q \in \delta(r, a) \text{ for some } r \in R\}.$$

Equivalently,

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a).$$

3. $q_0' = \{q_0\}$.

4. $F' = \{R \in Q' \,|\, \exists r \in R \; r \in F\}$.

Converting a NFA into a DFA

Let us take care of ε-transitions. For $R \subseteq Q$ (or equivalently $R \in Q'$) let

$$E(R) = \{q \,|\, q \text{ is reachable from } R \text{ through a number of } \varepsilon\text{-transitions.}\}$$

(note that $R \subseteq E(R)$ - why?). Modify now the construction as follows.

1. $Q' = \mathscr{P}(Q)$.
2. For $R \in Q'$ and $a \in \Sigma$ let $\delta'(R,a) = \{q \in Q \,|\, q \in E(\delta(r,a)) \text{ for some } r \in R\}$.
   Equivalently, $\delta'(R,a) = \bigcup_{r \in R} E(\delta(r,a))$.
3. $q_0' = E(\{q_0\})$.
4. $F' = \{R \in Q' \,|\, \exists r \in R \; r \in F\}$.

Converting a NFA into a DFA

We have proven the following

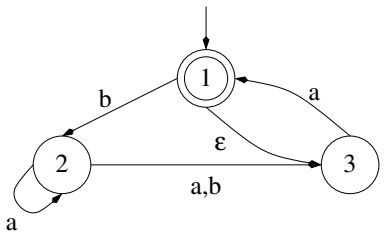
Theorem. *Every NFA has an equivalent DFA.*

which has the following

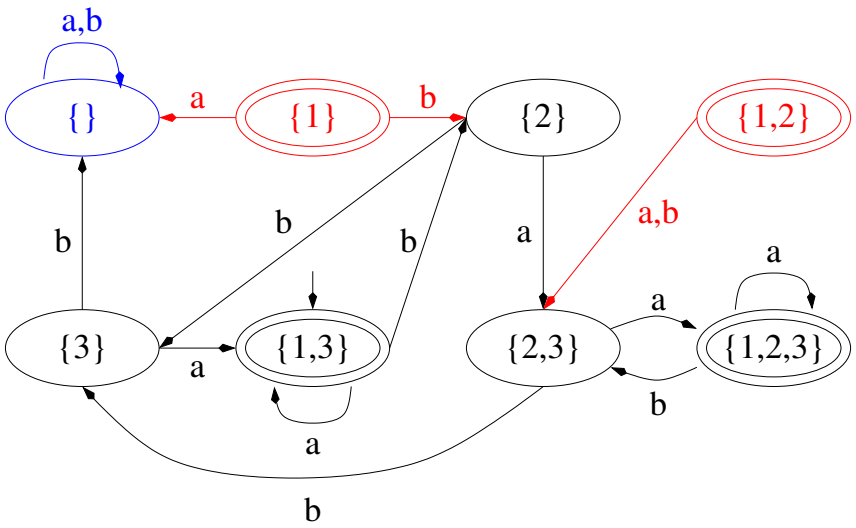
Corollary. *A language is regular iff some NFA recognises it.*

Example

Transform the following NFA into a DFA.



Having performed the transformation, the result is the following DFA:



We can remove the redundant states (and transitions). Observe that the {} state is really needed!

Closure under Regular Operations

- Union (already proven via Cartesian-product construction; we will see an alternative proof)

$$A \cup B = \{x \,|\, x \in A \text{ or } x \in B\}$$

- Concatenation

$$A \circ B = \{xy \,|\, x \in A \text{ and } y \in B\}$$

- Star

$$A^* = \{x_1 x_2 \ldots x_k \,|\, k \geq 0 \text{ and } x_i \in A \text{ for every } i, 1 \leq i \leq k\}$$

Proofs by pictures: Union

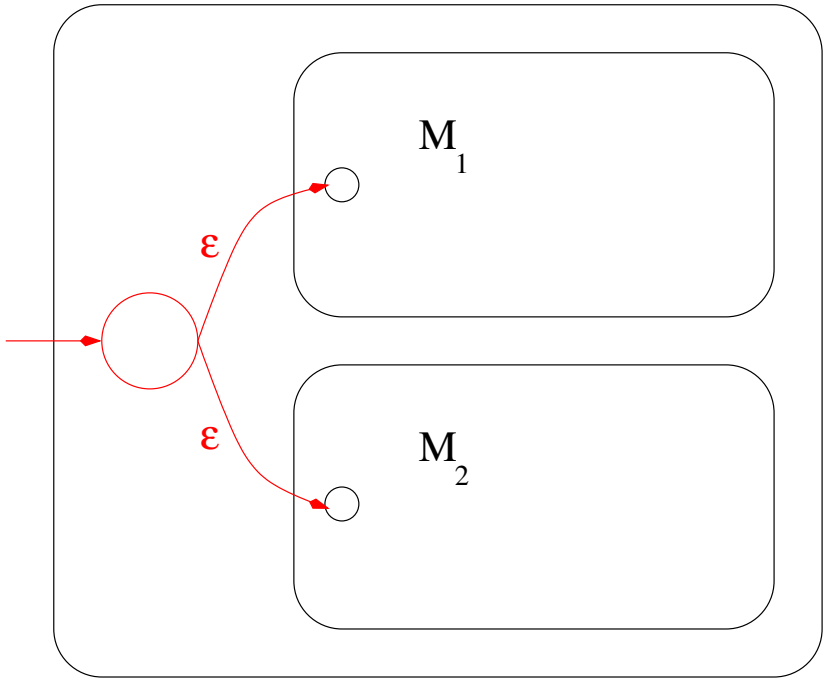


Figure 6: $M_1 \cup M_2$

Proofs by pictures: Concatenation

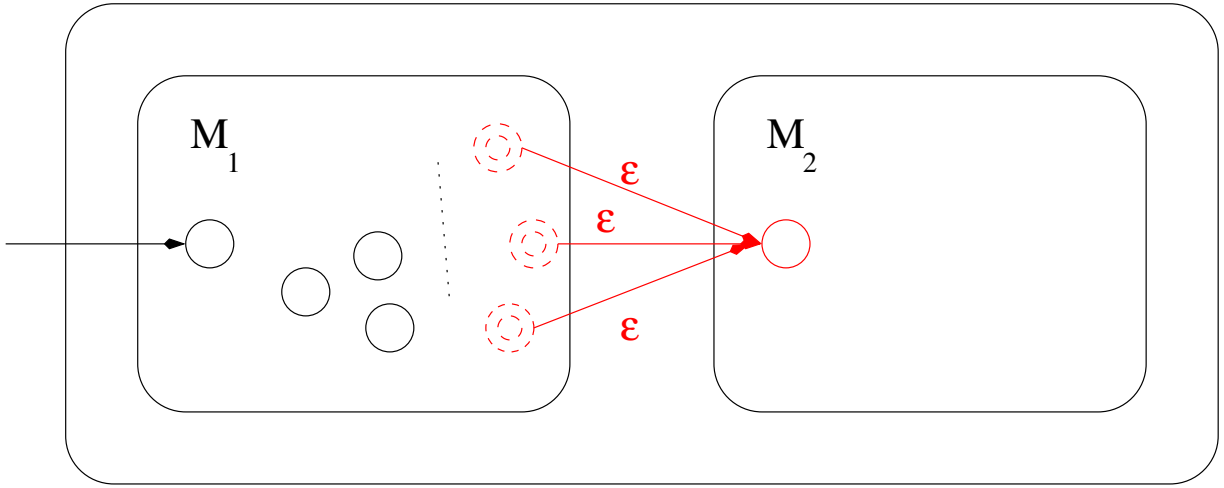


Figure 7: $M_1 \circ M_2$

Proofs by pictures: Star

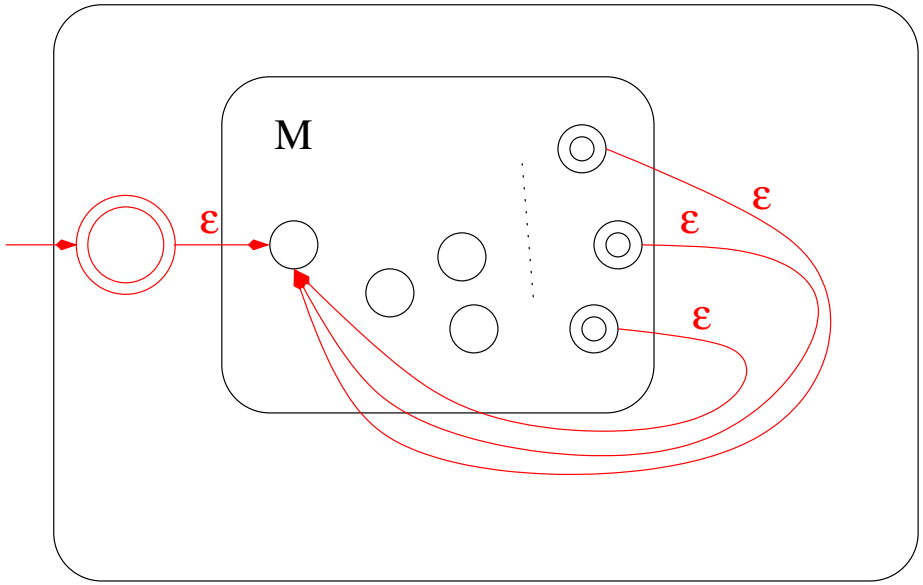


Figure 8: M^*

Equivalence of FA and Regular Expression (RE)

Theorem. *A language is regular if and only if some Regular Expression describes it.*

Proof:

$\Leftarrow$ **Given a RE R, construct a NFA that recognises $L(R)$**: easy, by **Structural Induction**:

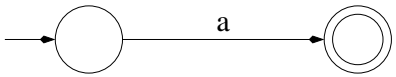
Base case: All initial REs can be implemented by NFAs (easily checked).

Inductive step: We have already proven that the class of regular languages is closed under regular operations.

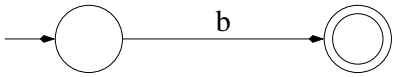
$\Rightarrow$ **Given a DFA M, construct a RE that recognises $L(M)$**: not so easy.

Example: $(ab \cup a)^*$

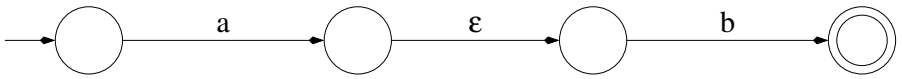
a



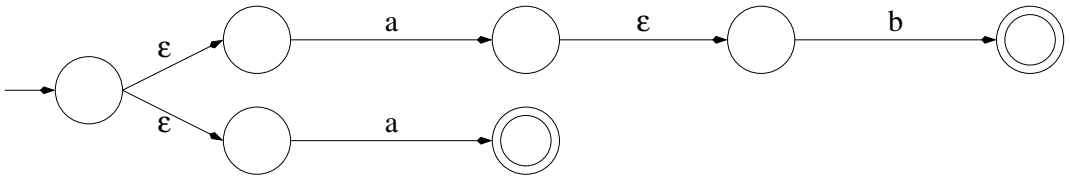
b



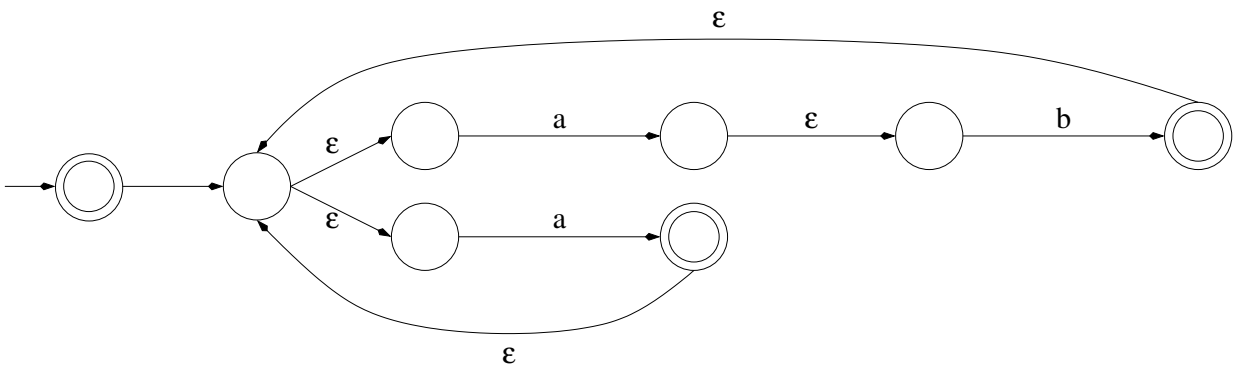
ab



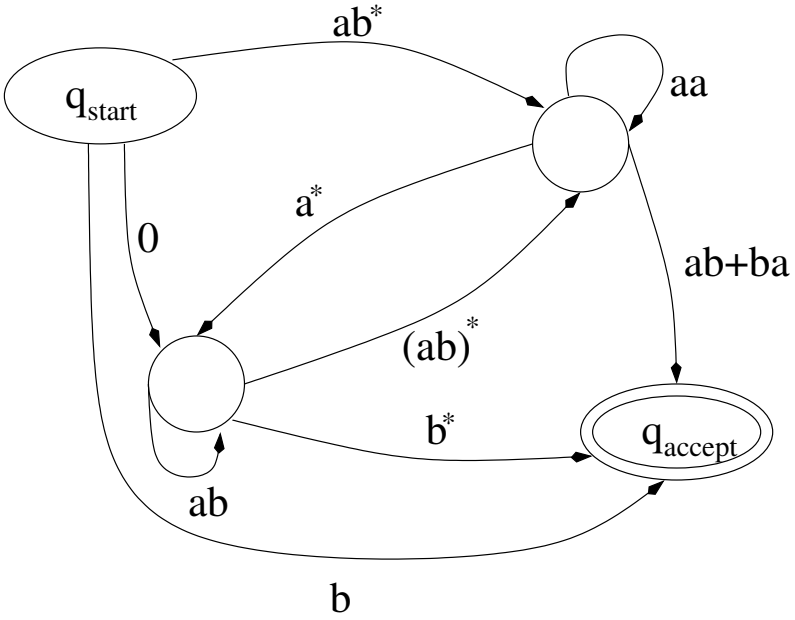
$ab \cup a$



$(ab \cup a)^*$



Generalised Non-deterministic Finite Automaton (GNFA)

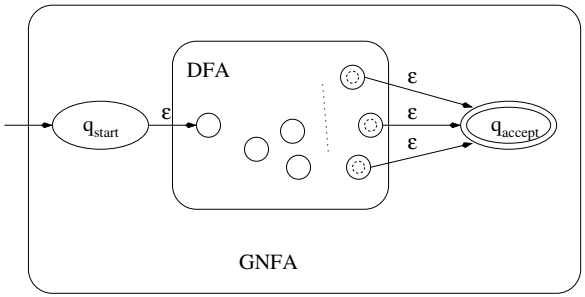


1. A single start state with no incoming edges.
2. A single accept state with no outgoing edges.
3. Every transition is labelled with a regular expression.

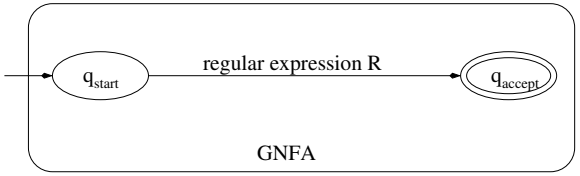
See the **formal definition** in the text-book.

DFA into RE

1. Convert the given n-state DFA (NFA) into an **equivalent** $n+2$-state GNFA.



2. **Eliminate** all the **internal states** of the GNFA **one by one**, while **preserving** the **language** recognised by the automaton.

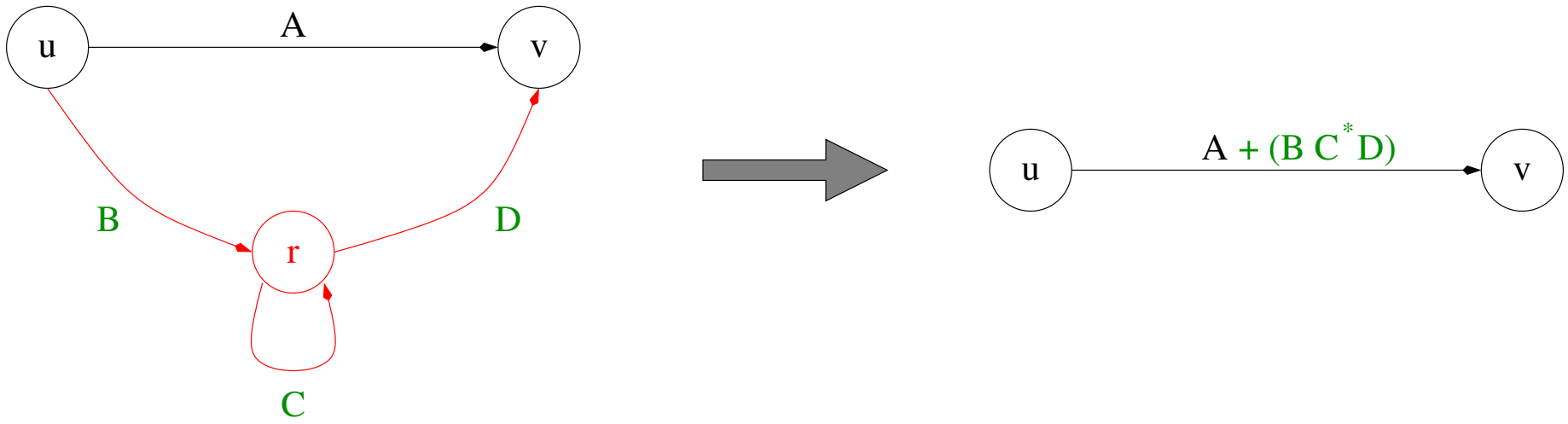


3. We are done, as $L(R) = L(\mathrm{DFA})$!

Internal State Elimination

Pick an internal state r for elimination.

For every two states u and v, $u \neq r \neq v$, do the following:



(note that we have to consider the case $u \equiv v$, too!).

Non-Regular Languages

We know how to prove that a given language L is regular:

- Construct a DFA (or even an NFA) that recognises L.
- Produce a Regular Expression that describes L.

How do we prove that a given language N is not regular?

- Cannot be recognised by any DFA: assume there is a DFA that recognises N and then use an Adversary Argument against that DFA in order to **get a contradiction**.

Example: DFAs cannot count

Question: Prove that the language $L = \{a^n b^n \,|\, n \in \mathbb{N}\}$ is not regular.

Proof: Assume, for the sake of contradiction, there is a DFA M that recognises L.

Suppose M has m states. Now consider the set of words $S = \{a^n \,|\, 0 \leq n \leq m\}$. There are $m+1$ words in S, so by the *Pigeon-Hole Principle* there must be two different words from S, a^i and a^j ($i \neq j$), that end up into the same state q (when started from the start state of M).

Now continue from q with the word b^i: $a^i b^i$ and $a^j b^i$ both end up into the same state q'. But $a^i b^i \in L$ while $a^j b^i \notin L$, so q' is an accept state and a non-accept state at the same time: a contradiction. $\square$

The Pumping Lemma is a generalisation of the above. It **abstracts away** the *DFA stuff* as well as the *adversary argument*.

Pumping Lemma

Lemma. *For every regular language L, there is a number p (called "pumping length" of L) such that every word $w \in L$ of length at least p can be divided into three parts, $w = xyz$, satisfying the following conditions:*

1. $xy^iz \in L$ *for every* $i \in \mathbb{N}$ *(in particular* $xz \in L$ *as one can take* $i = 0$*),*
2. y *is a non-empty string, and*
3. *the length of the string* xy *is not greater than* p*.*

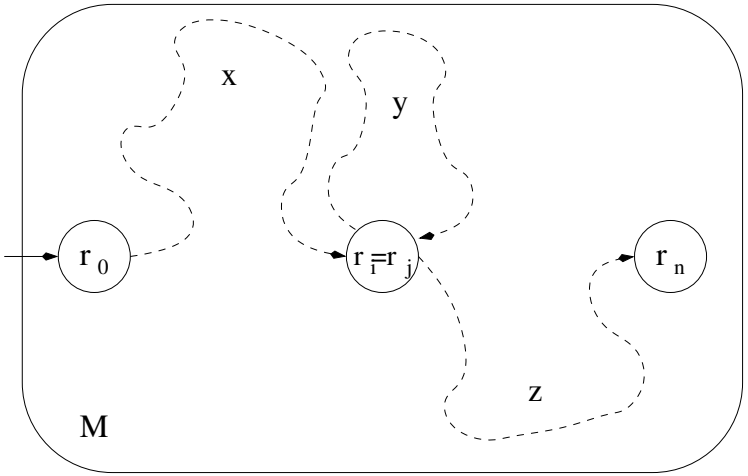
Pumping Lemma

Proof: Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA that recognises L, and let us set the pumping length p to be the number of states in M: $p = |Q|$.

Consider the computation of M on the word $w = w_1 w_2 \ldots w_n$ (note that $n \geq p$):

$$q_0 = r_0 \rightarrow^{w_1} \rightarrow r_1 \rightarrow^{w_2} \rightarrow r_2 \rightarrow^{w_3} \rightarrow \cdots\cdots\cdots r_{n-1} \rightarrow^{w_n} \rightarrow r_n \in F.$$

Let i be the first moment in time when we see a repetition in the sequence of states$r_0, r_1, r_2, \ldots r_n$ (such an i must exists by the Pigeon-Hole Principle, as $n+1 > p$): i.e. $r_i = r_j$ for some $j < i$ and the states $r_0, r_1, r_2, \ldots r_{i-1}$ are all different (note that this implies $i \leq p$).
We can then set
$x = w_1 w_2 \ldots w_j$, $y = w_{j+1} \ldots w_i$, and $z = w_{i+1} \ldots w_n$.



It is now straightforward to check that all three conditions are met. □

Applications[1]

Example 1: The language $L = \{0^n 1^n \,|\, n \in \mathbb{N}\}$ is not regular.

Take the word $0^p 1^p \in L$ where p is the pumping length of L. If $0^p 1^p = xyz$ and $|xy| \leq p$, then $x = 0^i$, $y = 0^j$ and $z = 0^{p-i-j} 1^p$. But then $xy^2z = 0^{p+j}1 \notin L$ as $|y| > 0$, i.e. $j > 0$.

Example 2: The language $C = \{w \,|\, w \text{ has an equal number of 0's and 1's}\}$.

Take the word $0^p 1^p \in C$ where p is the pumping length of C. Proceed as in the previous example.

Example 3: The language $F = \{ww \,|\, w \in \{0,1\}^*\}$.

Take the word $0^p 1 0^p 1 \in F$ where p is the pumping length of F. Note that $x = 0^i$, $y = 0^j$, and $z = 0^{p-i-j} 1 0^p 1$. Observe that $xy^2z \notin F$.

Example 4: The set P of palindromes over $\{0,1\}$ is not a regular language.

Take the word $0^p 1 0^p$ where p is the pumping length of P.

[1] **There are unnecessary complications in the text-book.**

More Applications

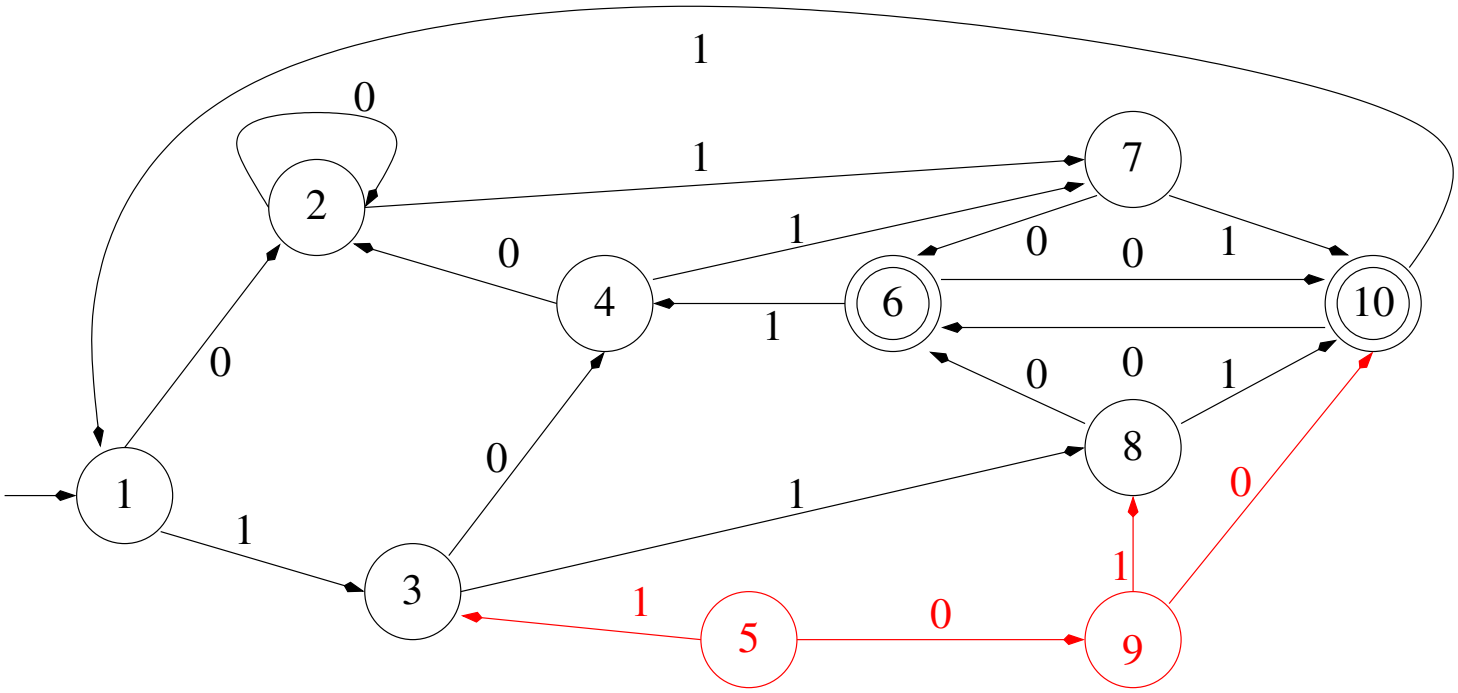
Example 5: The (*unary*) language $S = \left\{ 1^{n^2} \,|n \in \mathbb{N} \right\}$ is not regular.

Take the word 1^{p^2} where p is the pumping length of S. Now $x = 1^i$, $y = 1^j$, and $z = 1^{p^2-i-j}$. Observe that $i + j \leq p$ and therefore $j \leq p$. Consider xy^2z. $\left|xy^2z\right| = p^2 + j$. But $p^2 < p^2 + j < (p+1)^2$ as $0 < j \leq p$, therefore $p^2 + j$ cannot be a perfect square.

Example 2: The language $E = \left\{ 0^i 1^j \,|i > j \geq 0 \right\}$ is not regular.

Need to pump it down! Take $0^{p+1}1^p$ where p is the pumping length of E. We have $x = 0^i$, $y = 0^j$, and $z = 0^{p+1-i-j}1^p$ where $j > 0$. Consider xy^0z, i.e. $xz = 0^{p+1-j}1^p$. But $j \geq 1$, so $p + 1 - j \leq p$.

DFA Minimisation: Example



Find a DFA that recognises the **same language** as the DFA above does, and has **as few states as possible**.

There are finitely many possibilities, but we need be able to decide whether **two** given **DFAs** recognise the **same language**.

DFA Minimisation

An algorithm that "minimises" any given DFA, i.e. finds an equivalent DFA with the minimal number of states. It answers at once the following important questions:

1. What is the smallest (in terms of number of states) DFA that recognises the same language?

2. Given two DFAs, do they recognise the same language (if both **DFAs** are **minimal** then they are **isomorphic**, i.e. there is a **normal form DFA** that recognises a given regular language)?

Outline of the algorithm

1. Remove all the states that are not reachable from the start state.

2. Contract a set of states that are equivalent into a single state.

Two states, s and t, are **equivalent** if any word w, which is accepted when starting from s, is accepted when starting from t and vice-versa (in other words, there is no word that is accepted when starting from one of the states, but rejected when starting from the other).

Equivalent and Distinguishable states

Definition. *(*Extended Transition Function*) Let a DFA $M = (Q, \Sigma, \delta, q_0, F)$ be given, and let $\hat{\delta} : Q \times \Sigma^* \to Q$ be defined as*

$$\begin{aligned} \hat{\delta}(s, \varepsilon) &= s \\ \hat{\delta}(s, w_1 w_2 \ldots w_n) &= \hat{\delta}(\delta(s, w_1), w_2 \ldots w_n) \end{aligned}$$

for every state s and every word $w_1 w_2 \ldots w_n$.

Question: What is $\left\{ w \in \Sigma^* \,\middle|\, \hat{\delta}(q_0, w) \in F \right\}$?

Definition. *(Equivalent states) Two states s and t are equivalent if*

$$\left\{ w \in \Sigma^* \,\middle|\, \hat{\delta}(s, w) \in F \right\} = \left\{ w \in \Sigma^* \,\middle|\, \hat{\delta}(t, w) \in F \right\}.$$

Otherwise, s and t are distinguishable, i.e. there is a witness word u such that either $\hat{\delta}(s, u) \in F$, $\hat{\delta}(t, u) \notin F$ or $\hat{\delta}(s, u) \notin F$, $\hat{\delta}(t, u) \in F$.

Team-Splitting Algorithm

Best explained in terms of teams of players (sets of states) that pass balls of different colours (input symbols via transitions)[2].

Start Disqualify the useless players (states that are unreachable from the start state).

Round 0 Start with two teams, one consisting of all the accept states and the other consisting of all the non-accept states. Set $i := 1$.

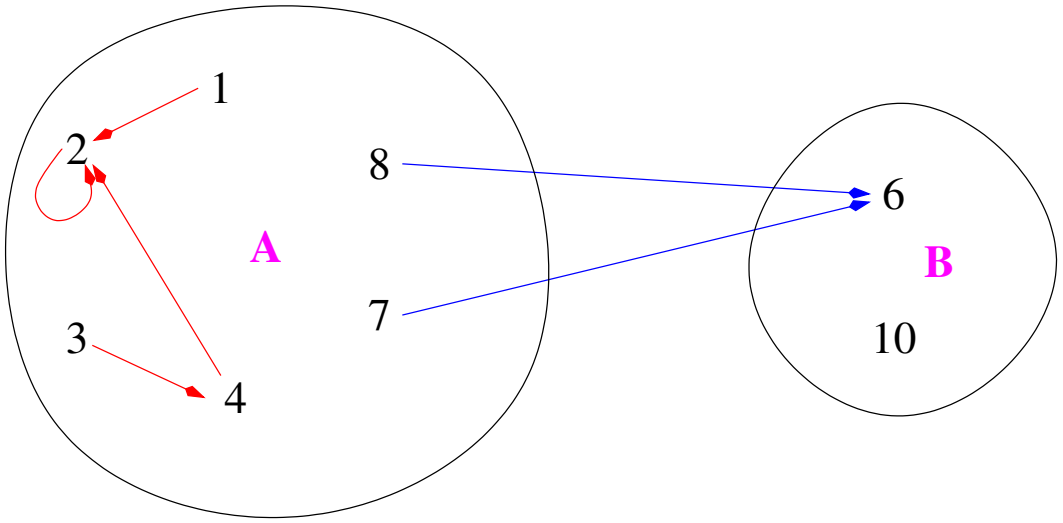
Round i For every team $S \in \mathscr{P}(Q)$ and every colour $a \in \Sigma$, check if all players from S agree on passing an a-ball. If not, split the team S into the maximal sub-teams that agreed, set $i := i + 1$, and go to **Round** i.

End Return the teams as sets of equivalent states.

[2] Following a presentation by Zeph Grunschlag from Columbia University.

Example: round $0 \to 1$

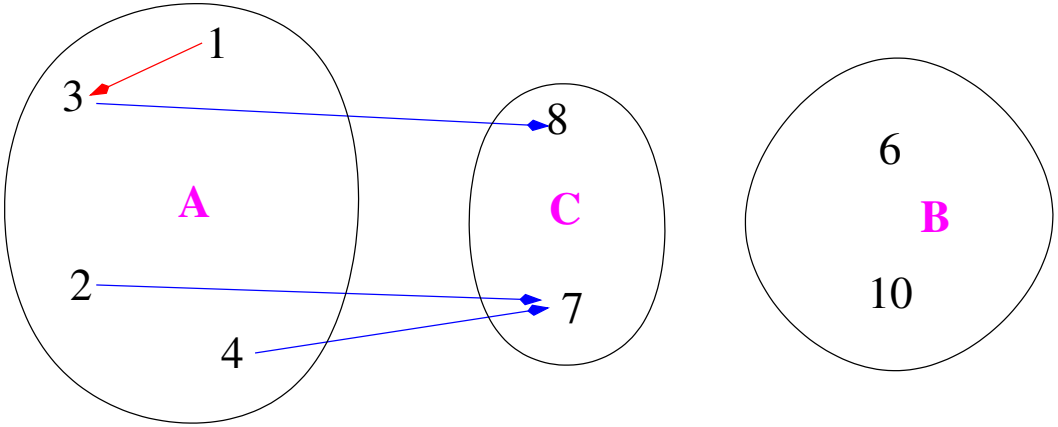
Disqualified the players 5 and 9. Created the two round-0 teams, $A = \{1,2,3,4,7,8\}$ and $B = \{6,10\}$.



Split the team A on a 0-ball.

Example: round $1 \to 2$

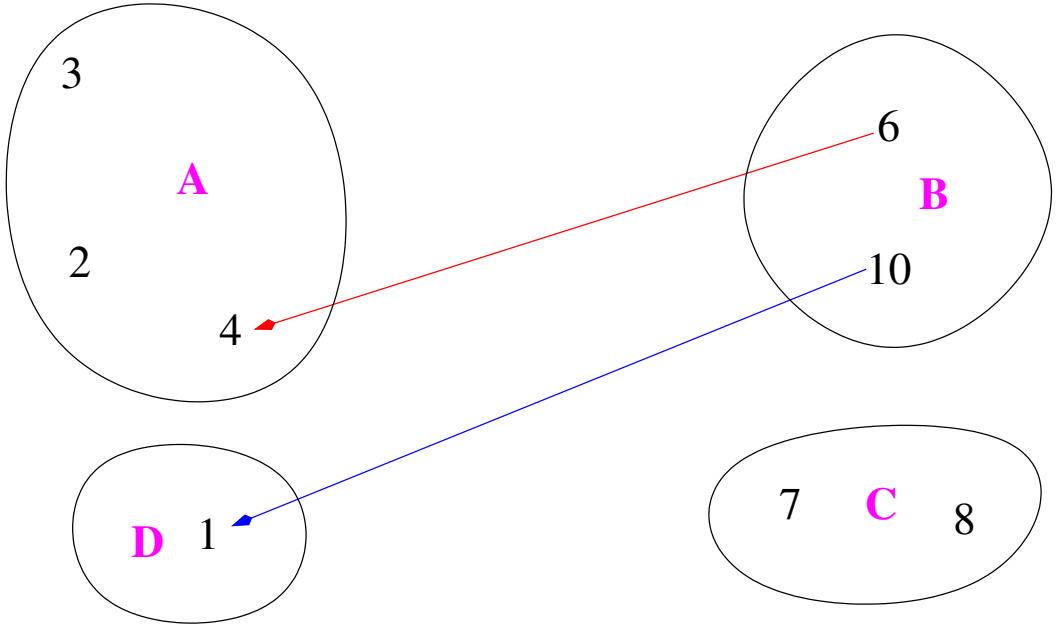
Split A into two new teams, $A = \{1,2,3,4\}$ and $C = \{7,8\}$.



Split the team A on a 1-ball.

Example: round $2 \to 3$

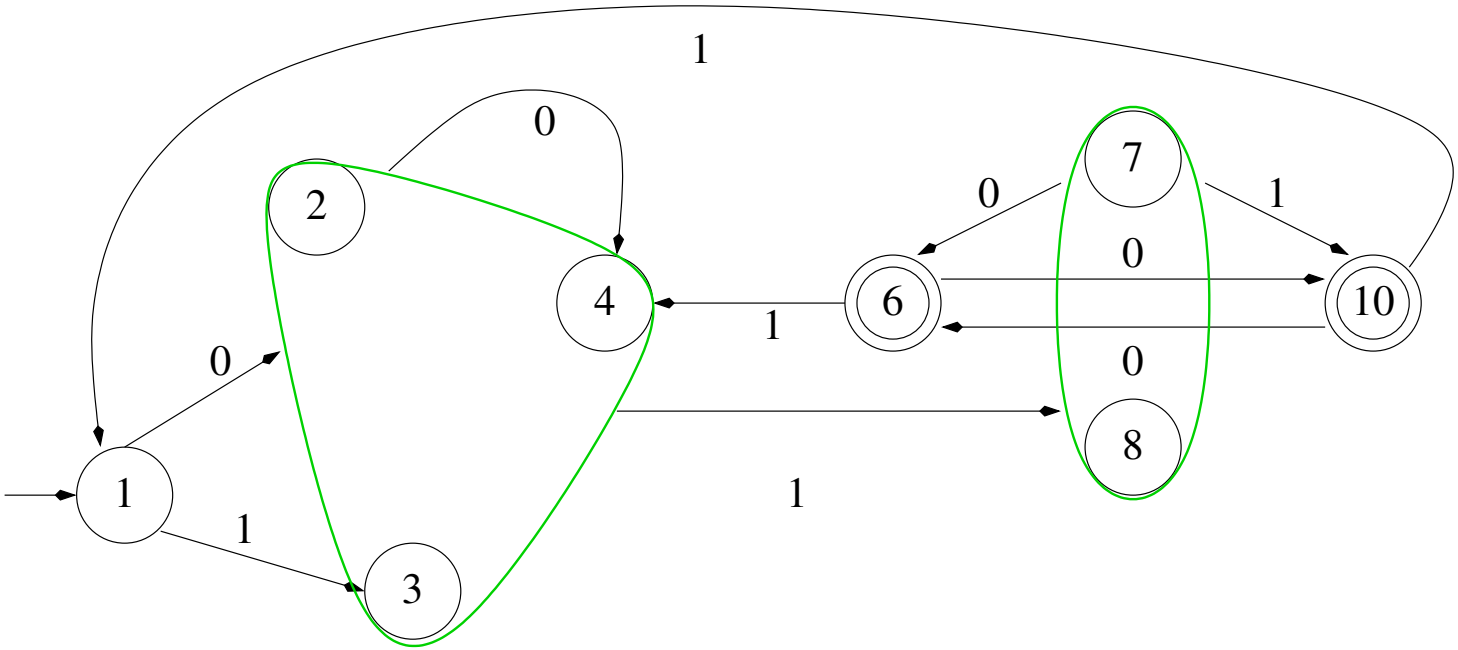
Split A into two new teams, $A = \{2,3,4\}$ and $D = \{1\}$.



Split the team B on a 1-ball. Now every team agree on every ball!

Example Minimised

Therefore, $2 \equiv 3 \equiv 4$ and $7 \equiv 8$.



Correctness of the algorithm

Proposition. *The Team-Splitting algorithm terminates.*

Proof. There cannot be more than $n-1$ splits (where n is the number of players). □

Proposition. *Every two players that are in the same team in the end are equivalent.*

Proof. Pick any sequence of passes (a word) and prove, by induction of its length, that it cannot tell the two players apart. □

Proposition. *Every two players that have been split at some round cannon be equivalent.*

Proof. Induction on the round number. The inductive step - a proof by contradiction. □

Equivalence and Isomorphism of DFAs

Question: Given two DFAs, M_1 and M_2, do they recognise the same language?

Solution: Put them **together**, **apply** the **Minimisation algorithm** as they were a single automaton, and see whether the **two start states are equivalent.**

Corollary. *If M_1 and M_2 are two minimal DFAs that recognise the same language, the minimisation algorithm always produces a bijection between the (equivalent) states of M_1 and M_2. We say that M_1 and M_2 are **isomorphic**.*

Proof. Build the bijection starting from the two start states. □

Finally

Theorem. *The Team-Splitting algorithm finds the minimal DFA equivalent to the original one.*

Proof. By first arguing that the DFA produced by the algorithm is equivalent to the minimal one, and then observing that the former cannot have unreachable or equivalent states, so it must be isomorphic to the latter. □